**5. Gradient Descent and XGBoost** **Date:18/09/2026**

Write a python program to predict a non-linear target variable **y** using two predictors $\mathbf{x_1}$ and $\mathbf{x_2}$.

$Y = 3x_1^2 + 2\sin(x_2) + €$

Where $€ \sim N(0, 2^2)$ is a random noise.

- Implement Multiple Linear Regression using Gradient Descent (not using built-in libraries like LinearRegression).
- Compare its performance with a nonlinear machine learning model (XGBoost) on the same dataset.

**Procedure:**

Step 1: Import the Required Libraries

Import the libraries needed for numerical calculations, data visualization, data splitting, feature scaling, performance evaluation, and XGBoost.

Step 2: Generate the Dataset

Set the random seed to 42. Generate 500 random values for the predictors x1 and x2 within the range -5 to 5.

Step 3: Generate the Target Variable

Calculate the nonlinear target variable using:

y=3x12+2sin (x2)+ϵy = 3x_1^2 + 2\sin(x_2) + \epsilon y=3x12+2sin(x2)+ϵ

The random noise follows a normal distribution with mean 0 and variance $2^2$.

Step 4: Prepare the Input Data

Combine the two predictors, x1 and x2, into one dataset. The target variable is y.

Step 5: Split the Dataset

Divide the dataset into training data and testing data. Use 70% of the data for training and 30% for testing.

Step 6: Standardize the Predictors

Standardize x1 and x2 so that both predictors have a similar scale. This helps the Gradient Descent algorithm work efficiently.

Step 7: Apply Multiple Linear Regression

Use Multiple Linear Regression with Gradient Descent. Initialize the weights and intercept, calculate predictions, determine the error, and update the weights repeatedly.

Step 8: Train the Model

Use a learning rate of 0.05 and train the model for 1000 epochs. Record the loss during each epoch.

Step 9: Predict Using Linear Regression

Use the trained Linear Regression model to predict the target values for the test dataset.

Step 10: Evaluate Linear Regression

Evaluate the Linear Regression model using Mean Squared Error and $R^2$ Score.

Step 11: Apply XGBoost Regression

Create an XGBoost regression model to learn the nonlinear relationship between x1, x2, and y.

Step 12: Train the XGBoost Model

Train the XGBoost model using the same training dataset used for Linear Regression.

Step 13: Predict Using XGBoost

Use the trained XGBoost model to predict the target values for the test dataset.

Step 14: Evaluate XGBoost

Evaluate the XGBoost model using Mean Squared Error and $R^2$ Score.

Step 15: Compare Both Models

Compare the results of both models. The model with a lower Mean Squared Error and higher $R^2$ Score is considered better.

Step 16: Plot the Results

Display the Gradient Descent loss curve and compare the actual target values with the predictions from both models.

Step 17: Interpret the Results

Because the target variable has a nonlinear relationship with the predictors, XGBoost is expected to perform better than Multiple Linear Regression.

**Solution:**

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from xgboost import XGBRegressor

# Generate nonlinear dataset

np.random.seed(42)
n = 500

x1 = np.random.uniform(-5, 5, n)
x2 = np.random.uniform(-5, 5, n)

# True nonlinear target

y = 3 * (x1 ** 2) + 2 * np.sin(x2) + np.random.normal(0, 2, n)

X = np.vstack((x1, x2)).T  # shape should be (n, 2) instead of (2, n)

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Linear Regression via Gradient Descent

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Add intercept (bias term)

Xb_train = np.c_[np.ones(X_train_scaled.shape[0]), X_train_scaled]
```

```
Xb_test = np.c_[np.ones(X_test_scaled.shape[0]), X_test_scaled]

# Initialize weights

w = np.zeros(Xb_train.shape[1])

# Hyperparameters

alpha = 0.05
epochs = 1000
m = len(y_train)
losses = []

for epoch in range(epochs):
    # Predictions

    y_pred = np.dot(Xb_train, w)

    # Gradient

    grad = (1 / m) * np.dot(Xb_train.T, (y_pred - y_train))

    # Update weights

    w -= alpha * grad

    # Loss

    loss = (1 / (2 * m)) * np.sum((y_pred - y_train) ** 2)
    losses.append(loss)

# Predictions on test data

y_pred_lr = np.dot(Xb_test, w)
mse_lr = mean_squared_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

# Convert weights back to original scale

beta1 = w[1] / scaler.scale_[0]
beta2 = w[2] / scaler.scale_[1]
intercept = w[0] - (beta1 * scaler.mean_[0] + beta2 * scaler.mean_[1])

# XGBoost

xgb_model = XGBRegressor(
    n_estimators=200,
    learning_rate=0.1,
    max_depth=4,
    reg_lambda=1,
    random_state=42
)

xgb_model.fit(X_train, y_train)

y_pred_xgb = xgb_model.predict(X_test)
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)
```

```
# Results

print("Linear Regression (Gradient Descent):")
print(f"Equation: y = {intercept:.3f} + {beta1:.3f}*x1 + {beta2:.3f}*x2")
print(f"Test MSE: {mse_lr:.4f}, R²: {r2_lr:.4f}\n")

print("XGBoost:")
print(f"Test MSE: {mse_xgb:.4f}, R²: {r2_xgb:.4f}")

# Plots

# Loss curve

plt.figure(figsize=(12, 4))
plt.plot(losses)
plt.title("Gradient Descent Loss Curve (Linear Regression)")
plt.xlabel("Epochs")
plt.ylabel("Loss (MSE)")
plt.show()

# Predictions vs Actual

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(y_test, y_pred_lr, alpha=0.6, color="red")
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    "k--"
)
plt.title("Linear Regression (Gradient Descent)")
plt.xlabel("Actual y")
plt.ylabel("Predicted y")

plt.subplot(1, 2, 2)
plt.scatter(y_test, y_pred_xgb, alpha=0.6, color="green")
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    "k--"
)
plt.title("XGBoost Predictions")
plt.xlabel("Actual y")
plt.ylabel("Predicted y")

plt.tight_layout()
plt.show()
```

Multiple Linear Regression Results

---

Weights: [2.57089585e+01 2.07725504e-02 2.04361124e-01]

MSE: 552.1570912755078

RMSE: 23.49802313547903

R2 Score: -0.00806486449273014

XGBoost Results

```
--------------
MSE: 8.904671680896714
RMSE: 2.98406965081191
R2 Score: 0.983742875364293

Model Comparison
----------------
                    Model         MSE       RMSE  R2 Score
0  Multiple Linear Regression  552.157091  23.498023 -0.008065
1                     XGBoost    8.904672   2.984070  0.983743
Multiple Linear Regression Results
-----------------------------------
Weights: [27.09518631 -0.12174017 -0.38801412]
MSE: 560.5498669409999
RMSE: 23.67593434145736
R2 Score: 0.009247283562503705

XGBoost Results
--------------
MSE: 15.53641932714897
RMSE: 3.9416264824497222
R2 Score: 0.9725399102561825

Model Comparison
----------------
                    Model         MSE       RMSE  R2 Score
0  Multiple Linear Regression  560.549867  23.675934  0.009247
1                     XGBoost   15.536419   3.941626  0.972540
Multiple Linear Regression Results
Weights: [2.57089585e+01 2.07725504e-02 2.04361124e-01]
MSE: 552.1570912755078
RMSE: 23.49802313547903
R2 Score: -0.00806486449273014

XGBoost Results
MSE: 8.904671680896714
RMSE: 2.98406965081191
R2 Score: 0.983742875364293

Model Comparison
                    Model         MSE       RMSE  R2 Score
0  Multiple Linear Regression  552.157091  23.498023 -0.008065
1                     XGBoost    8.904672   2.984070  0.983743
Multiple Linear Regression Results
Weights: [2.57089585e+01 2.07725504e-02 2.04361124e-01]
MSE: 552.1570912755078
RMSE: 23.49802313547903
R2 Score: -0.00806486449273014
```

XGBoost Results
MSE: 8.904671680896714
RMSE: 2.98406965081191
R2 Score: 0.983742875364293

Model Comparison

| | Model | MSE | RMSE | R2 Score |
|---|---|---|---|---|
| 0 | Multiple Linear Regression | 552.157091 | 23.498023 | -0.008065 |
| 1 | XGBoost | 8.904672 | 2.984070 | 0.983743 |

Linear Regression (Gradient Descent):
Equation: y = 26.881 + 0.085*x1 + -0.003*x2
Test MSE: 589.3888, $R^2$: -0.0037

XGBoost:
Test MSE: 6.2443, $R^2$: 0.9894







